SỔ TAY THỰC HÀNH • JAVA BACKEND

Xây dựng REST API quản lý công việc cá nhân

Lộ trình Spring Boot theo từng chặng, kèm kiến thức nền, cú pháp dùng trong dự án thực tế và checklist tự kiểm tra.

> Dự án xuyên suốt
> Task Manager API: người dùng đăng ký, đăng nhập, rồi quản lý các công việc của chính mình qua REST API.

Phù hợp khi học đồng thời khóa Spring REST API. Các ví dụ dùng Java hiện đại, Spring MVC, Spring Data JPA, Bean Validation và Spring Security.

Tài liệu được thiết kế để bạn tự code. Mỗi chặng có mục tiêu, kiến thức cần nắm, mẫu cú pháp và điều kiện hoàn thành.

Lộ trình học gợi ý: hoàn thiện User CRUD -> DTO và validation -> lỗi thống nhất -> Task và phân trang -> phân quyền theo chủ sở hữu -> JWT -> tài liệu và kiểm thử.

Cách dùng tài liệu

Mỗi chặng nên đi theo vòng lặp: xem bài học liên quan -> tự thiết kế API -> code -> gọi API bằng Postman -> ghi lại điều chưa hiểu -> sửa trước khi sang phần kế tiếp. Đừng dán toàn bộ ví dụ vào dự án cùng lúc; hãy hiểu vì sao mỗi lớp tồn tại.

Mục tiêu cuối

- REST API có đăng ký, đăng nhập và hồ sơ người dùng.
- Mỗi tài khoản chỉ truy cập được công việc thuộc về mình.
- JPA lưu User và Task; danh sách Task có phân trang.
- Input được validate; lỗi có status và nội dung nhất quán.
- Password được băm; API riêng tư được bảo vệ bằng bearer JWT.
- README và Postman collection giúp người khác chạy, hiểu và kiểm tra dự án.

Luồng request và trách nhiệm từng lớp

```
HTTP request
   -> Controller: nhận input, validate, chọn HTTP status
     -> Service: nghiệp vụ, transaction, quyền sở hữu
       -> Repository: truy vấn database
          -> Entity: mô hình dữ liệu lưu trữ
 <- DTO response: chỉ trả dữ liệu API cần công khai
```

Controller nên mỏng. Quy tắc nghiệp vụ và kiểm tra quyền không đặt trong controller hoặc repository. Repository không nên trả thẳng entity ra ngoài API.

Cấu trúc package gợi ý

```
com.example.taskmanager
  config/          SecurityConfig, cấu hình ứng dụng
  controller/      AuthController, UserController, TaskController
  dto/             request/ và response/
  entity/          User, Task, TaskStatus
  exception/       exception nghiệp vụ, handler lỗi
  repository/      UserRepository, TaskRepository
  security/        UserDetailsService, JWT support
  service/         AuthService, UserService, TaskService
```

> Lưu ý về bộ khung hiện tại
> Dự án hiện còn lớp chưa hoàn chỉnh và có đoạn code chưa biên dịch được. Chặng 1 bắt đầu bằng việc đưa skeleton về trạng thái build được; không cần giữ nguyên tên lớp/package mẫu nếu chúng gây cản trở.

CHẶNG 01 / SPRING BOOT • DI • JPA

Ổn định bộ khung và hoàn thiện User CRUD

Mục tiêu: hiểu request đi qua controller, service, repository thế nào; lưu User bằng JPA; tạo API theo từng bước. Bắt đầu với CRUD chưa có đăng nhập để tập trung vào nền tảng.

Kiến thức cần nắm

- Spring quản lý bean bằng IoC container; các lớp phụ thuộc nhau nên dùng constructor injection.
- Entity mô tả bảng; Repository cung cấp thao tác database; Service chứa nghiệp vụ; Controller ánh xạ HTTP.
- Không gọi repository trực tiếp từ controller khi nghiệp vụ bắt đầu có nhiều bước.
- Một thao tác ghi nhiều dữ liệu nên nằm trong transaction ở tầng service.

Cú pháp nên dùng

```
@Service
public class UserService {
    private final UserRepository userRepository;

    public UserService(UserRepository userRepository) {
        this.userRepository = userRepository;
    }

    @Transactional
    public UserResponse create(CreateUserRequest request) {
        // kiểm tra nghiệp vụ, đổi DTO thành entity, lưu và đổi lại response
    }

    @Transactional(readOnly = true)
    public UserResponse getById(Long id) { ... }
}
```

Entity và repository

```
@Entity
@Table(name = "users")
public class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 100)
    private String name;

    @Column(nullable = false, unique = true, length = 254)
    private String email;
}

public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmailIgnoreCase(String email);
    boolean existsByEmailIgnoreCase(String email);
}
```

API User giai đoạn đầu

- POST /api/users - tạo user; trả 201 Created.
- GET /api/users/{id} - lấy một user; trả 200 hoặc 404.
- PUT /api/users/{id} - thay đổi các trường được phép; trả 200.
- DELETE /api/users/{id} - xóa user; thống nhất trả 204 hoặc response 200.
- [ ] Ứng dụng chạy; User CRUD hoạt động; ID lạ trả 404; email trùng bị từ chối.

CHẶNG 02 / REST • BEAN VALIDATION

Thiết kế DTO và validate input

Mục tiêu: tạo hợp đồng API rõ ràng, bảo vệ dữ liệu nội bộ của entity và chặn input sai ngay ở ranh giới HTTP.

Quy tắc thiết kế

- Tạo DTO riêng cho request và response; không bind JSON trực tiếp vào JPA entity.
- Không bao giờ đưa password hoặc trường nội bộ vào response.
- Validation chủ yếu đặt trên request DTO vì đó là dữ liệu người dùng gửi.
- Email nên được trim và chuẩn hóa chữ thường trước khi lưu; unique constraint vẫn là lớp bảo vệ cuối.

Record DTO và annotation

```
public record RegisterRequest(
    @NotBlank @Size(max = 100) String name,
    @NotBlank @Email @Size(max = 254) String email,
    @NotBlank @Size(min = 8, max = 72) String password,
    @Size(max = 500) String address
) {}

@PostMapping
public ResponseEntity<UserResponse> create(
        @Valid @RequestBody CreateUserRequest request) {
    UserResponse created = userService.create(request);
    return ResponseEntity.status(HttpStatus.CREATED).body(created);
}
```

Record phù hợp với DTO bất biến khi dùng Java hiện đại. Nếu khóa học hoặc project đang dùng Java cũ, dùng class thường với constructor/getter. Lưu ý giới hạn password phải phù hợp thuật toán băm và chính sách dự án; BCrypt chỉ xử lý tối đa 72 byte.

Chuyển Entity sang Response DTO

```
public record UserResponse(Long id, String name, String email, String address) {
    public static UserResponse from(User user) {
        return new UserResponse(user.getId(), user.getName(),
                user.getEmail(), user.getAddress());
    }
}
```

Thứ tự xử lý tạo user

```
String email = request.email().trim().toLowerCase(Locale.ROOT);
if (userRepository.existsByEmailIgnoreCase(email)) {
    throw new EmailAlreadyUsedException(email);
}
User user = new User(request.name().trim(), email, encodedPassword, request.address());
return UserResponse.from(userRepository.save(user));
```

> Đừng chỉ dựa vào existsByEmail
> Hai request đồng thời vẫn có thể cùng vượt qua bước kiểm tra. Đặt unique constraint ở database và xử lý lỗi vi phạm constraint để đảm bảo không tạo email trùng.

- [ ] Request thiếu tên/email bị 400.
- [ ] Email không đúng định dạng bị 400.
- [ ] Response không có password dù password tồn tại trong entity.
- [ ] POST thành công trả 201 và body chứa resource vừa tạo.

CHẶNG 03 / EXCEPTION HANDLING

Chuẩn hóa lỗi và HTTP contract

Mục tiêu: client có thể phân biệt lỗi input, lỗi nghiệp vụ và lỗi server; mọi endpoint trả lỗi theo cùng một cấu trúc.

Dùng exception theo ngữ nghĩa nghiệp vụ

```
public class ResourceNotFoundException extends RuntimeException {
    public ResourceNotFoundException(String resource, Object id) {
        super(resource + " not found: " + id);
    }
}

public class EmailAlreadyUsedException extends RuntimeException {
    public EmailAlreadyUsedException(String email) {
        super("Email is already registered: " + email);
    }
}
```

Gom xử lý bằng Controller Advice

```
@RestControllerAdvice
public class ApiExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    ProblemDetail notFound(ResourceNotFoundException ex) {
        ProblemDetail problem = ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND, ex.getMessage());
        problem.setTitle("Resource not found");
        problem.setProperty("code", "RESOURCE_NOT_FOUND");
        return problem;
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ProblemDetail validation(MethodArgumentNotValidException ex) {
        // gom field + default message; trả 400
    }
}
```

Spring hỗ trợ ProblemDetail cho dạng lỗi theo chuẩn Problem Details. Có thể giữ ApiResponse hiện có nếu API đang dùng envelope; quan trọng là chọn một định dạng và áp dụng nhất quán, không trả ResponseEntity lồng nhau.

Bảng quyết định mã HTTP

Tình huống	Mã	Ghi chú
Tạo resource thành công	201	Có thể kèm Location header
Đọc/cập nhật thành công	200	Response chứa DTO
Xóa thành công	204	Không có response body
Request không hợp lệ	400	Chi tiết field lỗi
Chưa xác thực	401	Thiếu/sai token
Không được phép	403	Đã xác thực nhưng thiếu quyền
Không tìm thấy	404	Resource không tồn tại hoặc không thuộc user
Trùng dữ liệu	409	Unique email chẳng hạn

- [ ] Không trả exception class, SQL hay stack trace cho client.
- [ ] Validation trả danh sách field lỗi rõ ràng.
- [ ] Lỗi database bất ngờ được log phía server; client nhận thông báo an toàn.

CHẶNG 04 / JPA • REST

Thêm Task, quan hệ và phân trang

Mục tiêu: mỗi Task có đúng một owner, trạng thái và hạn hoàn thành; client có thể CRUD và đọc theo trang.

Entity và quan hệ

```
public enum TaskStatus { TODO, IN_PROGRESS, DONE }

@Entity
@Table(name = "tasks")
public class Task {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, length = 200)
    private String title;

    @Column(length = 2000)
    private String description;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false, length = 20)
    private TaskStatus status = TaskStatus.TODO;

    private LocalDate dueDate;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "owner_id", nullable = false)
    private User owner;
}
```

Dùng EnumType.STRING để database lưu tên trạng thái dễ đọc. Không dùng CascadeType.REMOVE từ Task lên User. Chỉ thêm quan hệ ngược OneToMany trên User nếu có use case cần; tránh serialize entity hai chiều thành vòng lặp.

DTO cho Task

```
public record CreateTaskRequest(
    @NotBlank @Size(max = 200) String title,
    @Size(max = 2000) String description,
    LocalDate dueDate
) {}

public record UpdateStatusRequest(@NotNull TaskStatus status) {}

public record TaskResponse(Long id, String title, String description,
                           TaskStatus status, LocalDate dueDate, Long ownerId) {}
```

Repository có pagination

```
public interface TaskRepository extends JpaRepository<Task, Long> {
    Page<Task> findAllByOwnerId(Long ownerId, Pageable pageable);
    Optional<Task> findByIdAndOwnerId(Long id, Long ownerId);
}
```

Service và endpoint

```
@Transactional(readOnly = true)
public Page<TaskResponse> listMyTasks(Long ownerId, Pageable pageable) {
    return taskRepository.findAllByOwnerId(ownerId, pageable)
            .map(TaskResponse::from);
}

@GetMapping
Page<TaskResponse> list(@RequestParam(defaultValue = "0") @Min(0) int page,
                        @RequestParam(defaultValue = "10") @Min(1) @Max(100) int size,
                        Authentication authentication) { ... }
```

Trong ứng dụng thực tế, cân nhắc trả DTO phân trang rõ ràng thay vì phụ thuộc trực tiếp vào JSON serialization của Page. Quy định thứ tự sort ổn định, ví dụ createdAt DESC rồi id DESC.

- [ ] Task mới luôn bắt đầu ở TODO.
- [ ] Status không được nhận chuỗi tùy ý.
- [ ] Page index bắt đầu từ 0; size có min/max.
- [ ] CRUD và danh sách Task hoạt động qua Postman.

CHẶNG 05 / AUTHORIZATION • DATA ACCESS

Ràng buộc quyền sở hữu dữ liệu

Mục tiêu: người dùng chỉ có thể thao tác trên tài nguyên mình sở hữu. Đây là quy tắc nghiệp vụ và bảo mật quan trọng nhất của dự án.

Truy vấn theo owner ngay ở database

```
private Task getOwnedTask(Long userId, Long taskId) {
    return taskRepository.findByIdAndOwnerId(taskId, userId)
        .orElseThrow(() -> new ResourceNotFoundException("Task", taskId));
}

@Transactional
public TaskResponse update(Long userId, Long taskId, UpdateTaskRequest request) {
    Task task = getOwnedTask(userId, taskId);
    task.setTitle(request.title().trim());
    task.setDescription(request.description());
    task.setDueDate(request.dueDate());
    return TaskResponse.from(task);
}
```

Nguyên tắc an toàn

- Không tin ownerId/userId do client gửi lên trong JSON hoặc query string.
- Lấy danh tính đã xác thực từ SecurityContext rồi chuyển thành ID nội bộ.
- Áp dụng lọc owner cho GET list, GET detail, PUT, PATCH và DELETE; không chỉ kiểm tra lúc tạo.
- Trả 404 cho ID không tồn tại hoặc không thuộc user để tránh dò tài nguyên.
- Không dùng findById rồi quên kiểm tra chủ sở hữu.

User API sau khi có xác thực

Đăng ký tạo user qua POST /api/auth/register. Các thao tác hồ sơ nên là GET, PUT, DELETE /api/users/me. Email có thể đổi nếu nghiệp vụ cho phép; dùng ID user ổn định trong subject JWT để đổi email không làm danh tính token bị lệch.

Kịch bản kiểm tra hai tài khoản

- [ ] Đăng ký A và B, đăng nhập để nhận hai token riêng.
- [ ] A tạo task X; A đọc và sửa được X.
- [ ] B gọi GET/PUT/PATCH/DELETE với ID của X đều không đọc hoặc thay đổi được X.
- [ ] Danh sách task của B không chứa task của A.

> Rủi ro thường gặp
> Nếu TaskResponse trả owner entity thay vì ownerId/owner summary, có thể gây lazy loading, vòng lặp JSON hoặc lộ dữ liệu không cần thiết. Luôn trả DTO có trường được chọn.

CHẶNG 06 / SPRING SECURITY

Đăng ký, đăng nhập và bảo vệ bằng JWT

Mục tiêu: password được băm; login xác thực credential; Spring Security tự xác minh chữ ký và thời hạn JWT ở các request tiếp theo.

Password encoder và UserDetailsService

```
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}

@Bean
UserDetailsService userDetailsService(UserRepository users) {
    return email -> users.findByEmailIgnoreCase(email)
        .map(user -> User.withUsername(user.getEmail())
            .password(user.getPassword())
            .roles("USER")
            .build())
        .orElseThrow(() -> new UsernameNotFoundException("User not found"));
}
```

Mã hóa password lúc đăng ký và không giải mã lại. Khi đăng nhập, AuthenticationManager so khớp password thô với hash đã lưu.

SecurityFilterChain cho REST API

```
@Bean
SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable()) // chỉ hợp lý với API stateless dùng bearer token
        .sessionManagement(session -> session
            .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(resource -> resource.jwt(Customizer.withDefaults()))
        .build();
}
```

Login và JWT

- POST /api/auth/register: validate -> chuẩn hóa email -> kiểm tra trùng -> encode password -> lưu -> trả user DTO.
- POST /api/auth/login: xác thực credential bằng AuthenticationManager -> phát access token.
- Request API riêng tư gửi Authorization: Bearer .
- JWT tối thiểu cần subject ổn định, issuer, issuedAt, expiresAt; không đưa password hoặc dữ liệu bí mật vào claim.
- Dùng thư viện/khả năng JwtEncoder và JwtDecoder của Spring Security; không tự viết ký/parse JWT.

Cấu hình secret

```
# application.properties - chỉ là cấu hình local mẫu
app.jwt.secret=${JWT_SECRET}
app.jwt.expiration=PT1H

# Đặt JWT_SECRET qua biến môi trường / secret manager.
# Không commit secret thật vào Git.
```

Nếu dùng HMAC, secret phải đủ mạnh và được giữ bí mật. Với production, cân nhắc khóa bất đối xứng hoặc identity provider. Không log access token. Access token có thời hạn ngắn; refresh token là hạng mục riêng, cần rotation, thu hồi và lưu trữ an toàn.

CHẶNG 07 / PRODUCTION READINESS

README, migrations và chất lượng dự án

Mục tiêu: người khác có thể clone, cấu hình, chạy và kiểm tra API; thay đổi schema được kiểm soát; cấu hình production không dựa vào máy cá nhân.

README nên có

- Mô tả chức năng và sơ đồ package.
- Yêu cầu Java, Maven, MySQL và cách tạo database.
- Danh sách biến môi trường: DB_URL, DB_USERNAME, DB_PASSWORD, JWT_SECRET.
- Cách chạy ứng dụng và cách import Postman collection.
- Bảng endpoint, request mẫu, response mẫu và lỗi thường gặp.
- Quyết định kỹ thuật chính: DTO, transaction, ownership, JWT.

Database schema trong môi trường làm việc

Khi học, Hibernate ddl-auto=update có thể giúp dựng nhanh schema. Khi dự án ổn định và cần cộng tác, dùng migration có version (Flyway hoặc Liquibase), rồi đặt ddl-auto=validate để ứng dụng kiểm tra mapping mà không tự âm thầm sửa schema.

```
# local learning profile (không dùng làm mặc định production)
spring.jpa.hibernate.ddl-auto=update

# profile có migration quản lý schema
spring.jpa.hibernate.ddl-auto=validate
spring.flyway.enabled=true
```

Các kiểm thử nên viết khi đã tới chặng tương ứng

- Service unit test: email trùng, user không tồn tại, đổi status.
- Repository integration test: truy vấn theo owner và phân trang.
- Controller/API test: validation, status code, JSON contract.
- Security test: không token -> 401; user khác -> không xem/sửa/xóa task.
- Không chỉ kiểm tra happy path; kiểm tra các trường hợp dữ liệu và quyền bị từ chối.

Definition of Done

- [ ] Ứng dụng chạy từ README trên máy sạch.
- [ ] Không có secret trong Git; không log password/token.
- [ ] Response chỉ chứa DTO và lỗi không lộ chi tiết nội bộ.
- [ ] Tất cả thao tác task lọc theo user hiện tại.
- [ ] Có kiểm thử tự động cho luồng quan trọng và test thủ công Postman.
- [ ] Git history chia theo thay đổi nhỏ, thông điệp commit mô tả mục đích.

Bản đồ API và thứ tự triển khai

API đề xuất

Method + path	Yêu cầu	Kết quả chính
POST /api/auth/register	Public	201 + user/token (theo lựa chọn thiết kế)
POST /api/auth/login	Public	200 + access token
GET /api/users/me	Bearer	Hồ sơ hiện tại
PUT /api/users/me	Bearer	Cập nhật hồ sơ
DELETE /api/users/me	Bearer	Xóa tài khoản
POST /api/tasks	Bearer	Tạo task cho user hiện tại
GET /api/tasks?page=0&size;=10	Bearer	Trang task của user
GET /api/tasks/{id}	Bearer	Chi tiết task thuộc user
PUT /api/tasks/{id}	Bearer	Cập nhật nội dung task
PATCH /api/tasks/{id}/status	Bearer	Đổi trạng thái
DELETE /api/tasks/{id}	Bearer	Xóa task thuộc user

Thứ tự code một endpoint

- 1. Chốt request, response và status HTTP.
- 2. Viết validation trên request DTO.
- 3. Viết method repository cần thiết.
- 4. Viết nghiệp vụ service và transaction.
- 5. Viết controller gọi service.
- 6. Viết exception mapping.
- 7. Gọi bằng Postman, kiểm tra cả thành công và lỗi.
- 8. Thêm test tự động rồi commit thay đổi.

Lộ trình commit gợi ý

1. chore: stabilize Spring Boot project skeleton
2. feat: implement user CRUD with request and response DTOs
3. feat: add validation and consistent API errors
4. feat: add paginated task CRUD
5. feat: enforce task ownership
6. feat: add registration, login, and JWT resource server
7. docs: add setup guide and Postman collection
8. test: cover auth, validation, and ownership rules

Checklist review theo chuẩn đi làm

Thiết kế và bảo trì

- Tên class/method biểu đạt nghiệp vụ; không có typo trong tên repository hoặc exception.
- Mỗi lớp có trách nhiệm rõ; controller không viết query hoặc business rule.
- DTO là hợp đồng API; entity là mô hình persistence, hai thứ không trộn.
- Dùng Optional cho kết quả có thể thiếu; chuyển thành lỗi nghiệp vụ ở service.
- Transaction đặt ở service; query chỉ đọc có thể đánh dấu readOnly.
- Không trả Page nếu serialization làm lộ hoặc tải quan hệ không cần thiết.

Bảo mật và dữ liệu

- BCrypt/PasswordEncoder cho password; không tự băm bằng SHA-256.
- Không tin ID user do client truyền để quyết định quyền sở hữu.
- Không trả password hash, token, secret hoặc thông tin exception nội bộ.
- Unique constraint và foreign key được phản ánh trong schema.
- Cấu hình khác nhau theo môi trường; production secret lấy từ secret store.
- Token có hạn, chữ ký được xác minh, endpoint công khai được giới hạn rõ.

Chất lượng API

- Dùng danh từ số nhiều cho resource path: /users, /tasks.
- GET không thay đổi dữ liệu; PUT thay nội dung; PATCH thay một phần.
- Status code phản ánh kết quả; lỗi có format và code ổn định.
- Validation có thông báo đủ để sửa request nhưng không lộ dữ liệu nhạy cảm.
- Pagination có thứ tự ổn định và giới hạn size.

Khi gửi code để review

Gửi branch/commit hoặc các file liên quan, kèm cách chạy, request mẫu và lỗi bạn chưa chắc. Nêu rõ bạn muốn review logic, bảo mật, cấu trúc, hiệu năng hay tất cả. Mình có thể rà soát theo từng endpoint và ưu tiên nhận xét theo mức độ ảnh hưởng.

> Không cần làm mọi thứ cùng lúc
> Hoàn thành từng chặng, giữ ứng dụng ở trạng thái chạy được, rồi mới mở rộng. Nếu khóa học giới thiệu cách khác, ghi lại lựa chọn và lý do để bạn so sánh thay vì chỉ chép cú pháp.

Tài liệu tham khảo chính thức

Các liên kết dưới đây là tài liệu Spring chính thức để tra cứu cú pháp chi tiết theo phiên bản dependencies trong pom.xml của dự án.

Spring Boot - Validation
https://docs.spring.io/spring-boot/reference/io/validation.html
Tích hợp Bean Validation và kiểm tra dữ liệu.

Spring Security - OAuth 2.0 Resource Server JWT
https://docs.spring.io/spring-security/reference/servlet/oauth2/resource-server/jwt.html
Xác minh bearer JWT và cấu hình resource server.

Spring Data JPA - Query Methods
https://docs.spring.io/spring-data/jpa/reference/repositories/query-methods-details.html
Tên phương thức truy vấn, Pageable và Sort.

Spring Data JPA - Core Concepts
https://docs.spring.io/spring-data/jpa/reference/repositories/core-concepts.html
Repository và phân trang.

Spring Framework - Declarative Transaction Management
https://docs.spring.io/spring-framework/reference/data-access/transaction/declarative.html
@Transactional và ranh giới transaction.

Ghi chú: Spring Boot/Security tiếp tục được cập nhật. Hãy dùng phiên bản được quản lý bởi Spring Boot parent/BOM trong pom.xml; không tự ghim từng dependency Spring sang các phiên bản khác nhau.

SPRING BOOT / TASK MANAGER API

Phụ lục thực chiến từ code dở đến API hoàn chỉnh

Swagger/OpenAPI, kiểm thử, bảo mật, migration và vận hành theo các tình huống thường gặp ở dự án backend.

01 **Hiện trạng**

Khung User API còn lỗi biên dịch; chưa có API hoạt động.

02 **Khoảng trống**

PDF cũ chưa có OpenAPI và chưa chỉ rõ ca kiểm thử.

03 **Đầu ra**

Tám bài tập với test, tiêu chí đạt và câu hỏi tự giải thích.

Phần gốc 13 trang được giữ nguyên; các trang sau là nội dung bổ sung.

Áp dụng cho repo hiện tại: Spring Boot 4.1.1, Java 25, Maven, Spring MVC, Data JPA, MySQL, Validation và Spring Security. Phần này bổ sung cho PDF *Lộ trình Spring Boot - Task Manager REST API* 13 trang. Học theo chặng; mỗi chặng chỉ xong khi code chạy, test đạt và bạn giải thích được quyết định chính.

1. Kiểm kê PDF và code hiện tại

Chủ đề	PDF cũ	Repo hiện tại	Việc cần bổ sung
REST, DI, DTO, validation, JPA	Có lộ trình và ví dụ	Mới có khung, còn lỗi biên dịch	Bài tập theo endpoint, tiêu chí đạt và test
Xử lý lỗi, transaction, ownership	Có nguyên tắc	Handler đặt ngoài vùng quét component; service dở	Bài tập lỗi, transaction, phân quyền hai tài khoản
JWT, RBAC, refresh token	JWT có một chặng; RBAC và refresh token chưa thành bài riêng	Chưa triển khai	Bài tập security theo cấp độ, test từ chối và thu hồi
Swagger/OpenAPI	**Chưa có**	Chưa có dependency/cấu hình	Thêm tài liệu API chạy được và kiểm tra hợp đồng
Unit, web, repository, integration test	Có danh sách loại test ở trang 10-11	Chỉ có **contextLoads()**	Ma trận test, ca biên, cơ sở dữ liệu MySQL thật bằng Testcontainers
Migration, cấu hình môi trường, CI, vận hành	Có nhắc ngắn	Chưa triển khai	Bài tập Flyway, profiles, secrets, Actuator, CI và README

Repo hiện không biên dịch được: **UserController** và **UserService** viết dở; **User** có **@NotBlank(** chưa đóng; **UserRepository.exexistsByEmail** sai tên; **@NoArgsConstructor** không tiêm được **final UserRepository**. **GlobalExceptionHandler** nằm trong **com.example.exception**, ngoài cây package **com.example.crud** của **@SpringBootApplication**; kiểu trả về còn lồng **ResponseEntity**. Hai DTO và **AuthController** rỗng. **application.properties** chưa có datasource. Không sửa mọi thứ một lần: hoàn thành từng bài và giữ build xanh.

2. Cách học và thứ tự

Một buổi làm một lát cắt dọc nhỏ: đọc mục tiêu → tự viết contract HTTP → viết test thất bại có ý nghĩa → code → chạy test → gọi API bằng HTTP client → ghi lại điều chưa hiểu. AI được dùng để hỏi vì sao test sai hoặc review diff sau khi đã tự thử. Mỗi bài có **đầu ra** (file/API cụ thể), **ca kiểm tra** và **câu hỏi tự giải thích**. Không cần áp lịch bốn tuần; hoàn thành theo tiêu chí đạt.

Luồng chính: **0. Build → 1. User API → 2. Lỗi/validation → 3. Task/JPA/Flyway → 4. Authentication/ownership → 5. RBAC/refresh → 6. OpenAPI → 7. Vận hành/CI**. Viết test ngay ở bài liên quan; bài 7 chỉ hoàn thiện bộ test và pipeline. Những mục như cache, message queue, microservices là phần mở rộng, không phải điều kiện để một Task API nhỏ đạt chất lượng.

Bắt đầu ngay với repo này

1. Chạy **./mvnw -q -DskipTests compile**, ghi lại **lỗi đầu tiên** thay vì sửa ngẫu nhiên tất cả file.
2. Sửa lần lượt **User.java** (annotation dở), **UserController.java** (khai báo method), **UserService.java** (constructor và method dở), rồi **UserRepository.java** (tên query method). Chạy lại compile sau mỗi thay đổi nhỏ.
3. Đưa **GlobalExceptionHandler** vào vùng quét của ứng dụng và bỏ kiểu **ResponseEntity** lồng; kiểm tra một lỗi 404 qua HTTP sau khi có endpoint.
4. Chuẩn bị MySQL/test datasource và biến môi trường theo README sẽ viết; chạy **./mvnw test**. Test **contextLoads()** chỉ chứng minh context khởi động, chưa chứng minh CRUD đúng.
5. Tạo **một** test đầu tiên **createUser_whenValid_returns201AndNoPassword**, rồi làm **POST /api/users** đủ để test xanh. Tiếp tục **createUser_whenDuplicateEmail_returns409**, **getUser_whenMissing_returns404** trước khi chuyển sang Task.

Giữ thay đổi thành các commit nhỏ. Nếu build còn đỏ vì một file viết dở, chưa cần thêm JWT, Swagger hay Docker vào code; các phần đó đã có bài riêng ở dưới.

Bài 0 - Cứu bộ khung và hiểu Dependency Injection

Đầu ra: **./mvnw -q -DskipTests compile** thành công; ứng dụng khởi động được với MySQL local hoặc test profile. Sửa cú pháp ở ba file đang dở; đổi repository method thành **existsByEmailIgnoreCase**; dùng constructor injection rõ ràng; đưa exception/handler vào package con của **com.example.crud** hoặc cấu hình quét có chủ đích. Chọn một kiểu response lỗi, không lồng **ResponseEntity**. Rà dependency: OAuth2 Authorization Server dùng để *phát hành* token theo OAuth2; đừng giữ starter này chỉ vì API sẽ *xác minh* bearer JWT.

Tự kiểm tra: build sạch; **contextLoads** chạy với test datasource; có thể chỉ ra bean nào được Spring tạo và service nhận repository bằng cách nào. **Câu hỏi:** Vì sao **final UserRepository** với **@NoArgsConstructor** không phải constructor injection?

Bài 1 - User API, DTO và HTTP contract

Đầu ra: **POST /api/users** (201), **GET /api/users/{id}** (200/404), **GET /api/users** (có page/size), **PUT /api/users/{id}** (200), **DELETE /api/users/{id}** (204). Tạo request DTO và response DTO; không nhận/trả JPA entity trực tiếp. Chuẩn hóa email trước khi lưu; database có unique constraint; response tuyệt đối không có password/hash. Sau khi thêm auth, thay API hồ sơ công khai bằng **/api/users/me** theo PDF gốc.

Test cần viết: **@WebMvcTest** + **MockMvc** cho status/JSON/validation; service unit test cho email trùng và ID không tồn tại; repository test cho email duy nhất và truy vấn không phân biệt hoa thường. Kiểm tra **POST** hợp lệ trả 201; tên rỗng trả 400; email trùng trả 409; ID lạ trả 404; JSON không có **password**. **Câu hỏi:** Controller, service và repository mỗi lớp giữ quyết định nào?

Bài 2 - Validation và lỗi nhất quán

Đầu ra: request DTO có **@NotBlank**, **@Email**, **@Size**, **@Valid**; lỗi dùng một contract duy nhất (khuyến nghị **ProblemDetail** với **code** ổn định). Xử lý **MethodArgumentNotValidException**, lỗi tham số trực tiếp nếu dùng method validation, 404, 409 và lỗi bất ngờ. Không đưa SQL, stack trace hay dữ liệu nhạy cảm ra client. Đừng dựa riêng vào **existsByEmail**: vẫn cần unique constraint và xử lý xung đột khi hai request đồng thời.

Test cần viết: mỗi loại lỗi kiểm tra HTTP status, **code**, thông điệp an toàn và field lỗi; xác nhận lỗi 500 được log phía server nhưng body không lộ exception. **Câu hỏi:** Vì sao email trùng có thể xảy ra dù service đã gọi **existsByEmail**?

Bài 3 - Task, JPA, pagination và migration

Đầu ra: **Task** có owner, title, description, status, dueDate; API CRUD + đổi status; truy vấn theo owner ngay trong repository; page size giới hạn và sort ổn định (**createdAt DESC, id DESC**). Viết Flyway **V1__create_users.sql**, **V2__create_tasks.sql** (nếu schema chưa có dữ liệu cần giữ); dùng **ddl-auto=validate** khi migration quản lý schema. Với MySQL, thêm **spring-boot-starter-flyway** và **org.flywaydb:flyway-mysql** theo tài liệu Boot; không dùng song song **schema.sql** để tạo cùng schema.

Test cần viết: repository test với MySQL Testcontainers và **@ServiceConnection**: A có 12 task, B có 2 task; trang 0 size 10 của A chứa 10 task, trang 1 chứa 2; B không thấy task A. Một full integration test kiểm tra migration chạy được trên database trống. Test **TODO** mặc định, status không hợp lệ trả 400, size > giới hạn trả 400. **Câu hỏi:** Vì sao **EnumType.STRING**, quan hệ LAZY và index **(owner_id, created_at, id)** hữu ích ở đây?

Bài 4 - Đăng ký, đăng nhập và quyền sở hữu

Đầu ra: **POST /api/auth/register**, **POST /api/auth/login**, **/api/users/me**, **/api/tasks**. Băm password bằng **PasswordEncoder**; không log password/token. Chỉ lấy user ID từ danh tính đã xác thực; không tin **ownerId** trong body/query. Cấu hình **SecurityFilterChain** rõ public/private endpoints. Thêm **spring-boot-starter-oauth2-resource-server** để xác minh bearer JWT; nếu tự phát JWT để học, dùng **JwtEncoder**/**JwtDecoder** của Spring Security, không tự viết parser/filter chữ ký. Secret/khóa lấy từ môi trường, không commit. Nếu dùng identity provider ngoài, API chỉ cần Resource Server và không cần tự xây Authorization Server.

Test cần viết: không token → 401; token hợp lệ → 200; A tạo task X; B đọc/sửa/đổi trạng thái/xóa X → 404; B list không chứa X; thay **ownerId** trong JSON không chiếm được task. Test password lưu là hash, không phải bản rõ; JWT hết hạn bị từ chối. **Câu hỏi:** Authentication khác authorization thế nào? Vì sao có thể trả 404 cho task của người khác?

Bài 5 - RBAC và refresh token (phần nâng cao của khóa học)

Đầu ra: hai role **USER**, **ADMIN**; thêm một endpoint quản trị nhỏ như **GET /api/admin/users**; đăng ký tự phục vụ luôn tạo **USER**, client không được chọn role. **USER** gọi admin endpoint → 403; **ADMIN** → 200. Thêm **POST /api/auth/refresh** và logout nếu muốn thực hành đúng chủ đề refresh token trong ảnh. Lưu **hash** refresh token, thời hạn và trạng thái thu hồi trong database; xoay vòng token khi refresh; refresh token cũ không dùng lại được; logout thu hồi token hiện hành. Thiết kế này là bài tập để hiểu vòng đời token, không phải bản thiết kế identity provider cho production.

Test cần viết: client gửi **role=ADMIN** khi register vẫn là USER; token USER không qua admin; refresh cũ bị từ chối sau rotation; token hết hạn/đã thu hồi bị từ chối. **Câu hỏi:** Vì sao RBAC không thay thế kiểm tra owner của từng Task?

Bài 6 - Swagger/OpenAPI như một hợp đồng API

Đầu ra: thêm **org.springdoc:springdoc-openapi-starter-webmvc-ui:3.2.0** (springdoc công bố dòng 3.1.x-3.2.x tương thích Boot 4.1.x); xác nhận **/v3/api-docs** và Swagger UI chạy với cấu hình security của dự án. Tài liệu hóa từng endpoint: request/response schema, status 200/201/204/400/401/403/404/409, ví dụ lỗi, page/size/sort và Bearer JWT security scheme. Không mô tả password/hash trong response. Cho phép UI/docs ở môi trường học; ở môi trường khác quyết định rõ phạm vi truy cập.

Bài thực hành: từ Swagger UI đăng ký → đăng nhập → Authorize bearer → tạo Task → liệt kê → thử không token. Export JSON OpenAPI để review; so sánh contract với kết quả MockMvc. **Test cần viết:** smoke test **/v3/api-docs** trả JSON và có **/api/tasks**; web test của từng endpoint xác nhận status/schema chính, vì tài liệu tự sinh không chứng minh hành vi chạy đúng. **Câu hỏi:** Trường nào trong DTO đổi sẽ làm client cũ hỏng? Bạn sẽ version hoặc giữ tương thích ra sao?

Bài 7 - Kiểm thử tự động, chạy dự án và vận hành

Ba lớp kiểm thử: (1) service unit test cho rule thuần; (2) web slice **@WebMvcTest** + **MockMvc** + **@MockitoBean** cho HTTP, validation và security; (3) **@DataJpaTest**/**@SpringBootTest** + MySQL Testcontainers cho query, mapping, migration và luồng xuyên tầng. Không viết test chỉ lặp lại code; ưu tiên ca lỗi, quyền và hợp đồng API. Lập bảng coverage theo use case, không chỉ phần trăm dòng.

Cấu hình: **application-local.properties** cho máy học, **application-test.properties** cho test, cấu hình production-like qua biến môi trường; dùng **@ConfigurationProperties** cho nhóm cấu hình riêng. Không commit mật khẩu/khóa. README ghi Java 25, cách chạy MySQL, biến môi trường, lệnh **./mvnw verify**, URL Swagger, tài khoản mẫu tạo thế nào và vài request **curl**.

Vận hành: thêm Actuator health/metrics; chỉ mở các management endpoint cần thiết và bảo vệ bằng security chain. Log có request/correlation ID, không chứa token/password; lỗi 5xx có log để truy tìm. Có Docker Compose cho MySQL local; một CI job dùng JDK 25 chạy **./mvnw --batch-mode verify**, bao gồm integration test với container, rồi đóng gói executable JAR. Dùng cùng lệnh verify ở máy cá nhân và CI.

Definition of Done: máy sạch có Java/Docker theo README chạy được dự án; migration dựng database trống; test tự động xanh; Swagger mô tả đúng endpoint; A không truy cập Task B; không có secret trong Git; health endpoint hoạt động và không công khai thông tin quản trị nhạy cảm.

3. Bài tổng hợp cuối khóa

Đề: Một người dùng tạo Task X, đổi trạng thái sang **IN_PROGRESS**, liệt kê theo trang, rồi đăng xuất. Một người khác không được nhìn thấy hoặc sửa X. Admin xem danh sách người dùng nhưng không tự động được sửa Task X nếu chưa có rule cho phép. Tài liệu OpenAPI phải hướng dẫn gọi toàn bộ luồng; CI chạy toàn bộ test.

Nộp để tự review: branch hoặc commit, README, OpenAPI JSON/link Swagger local, lệnh chạy test, bảng test case và ba quyết định kỹ thuật bạn tự giải thích: (1) ranh giới transaction, (2) quyền owner, (3) vòng đời token. Nếu không giải thích được, quay lại bài tương ứng dù test đang xanh.

4. Mở rộng sau khi lõi đã chắc

Chọn **một** hướng theo nhu cầu công việc: truy vấn hiệu năng (N+1, index, đo query), optimistic locking khi hai người sửa cùng Task, rate limiting cho login, cache dữ liệu đọc nhiều, tích hợp API ngoài có timeout/retry, hoặc async/event. Không thêm message broker hay microservices chỉ để làm dự án trông lớn hơn.

Tài liệu đối chiếu theo phiên bản

- Spring Boot testing: https://docs.spring.io/spring-boot/reference/testing/spring-boot-applications.html
- Spring Boot Testcontainers: https://docs.spring.io/spring-boot/reference/testing/testcontainers.html
- Spring Boot database initialization/Flyway: https://docs.spring.io/spring-boot/how-to/data-initialization.html
- Spring MVC validation: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-controller/ann-validation.html
- Spring MVC error responses: https://docs.spring.io/spring-framework/reference/web/webmvc/mvc-ann-rest-exceptions.html
- Spring Security JWT Resource Server: https://docs.spring.io/spring-security/reference/7.0/servlet/oauth2/resource-server/jwt.html
- Spring Security Resource Server starter: https://docs.spring.io/spring-security/reference/7.0/servlet/oauth2/index.html
- Springdoc compatibility/setup: https://springdoc.org/faq.html và https://springdoc.org/getting-started.html
- Spring Boot profiles/config: https://docs.spring.io/spring-boot/reference/features/profiles.html và https://docs.spring.io/spring-boot/reference/features/external-config.html
- Spring Boot Actuator: https://docs.spring.io/spring-boot/reference/actuator/endpoints.html
- GitHub Actions Maven: https://docs.github.com/en/actions/tutorials/build-and-test-code/java-with-maven